PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING | RR CAMPUS

Course: Software Engineering (UE24CS351A)	**Lab Session:** Lab 7 — Fuzzing for Input Validation Bugs
Student Name: Saathvik Shenoy Padubidri	**SRN:** PES1UG24CS394 (per1ug24cs394)
Section: 5th Sem, Section G	**Faculty:** Dr. Sapna V M
Date of Submission: November 11, 2026	**Assigned Script:** vending_machine.py / processor.py
GitHub Repository: https://github.com/saathviksp/L6_L7_30_vending_machine	**Repository Status:** Public (Verified on main)

1. Objective & Technical Background

The objective of this laboratory is to explore fuzz testing as a dynamic security and quality assurance technique by identifying and fixing latent input validation bugs in a vending machine processing module using the Python **Hypothesis** library. Unlike conventional unit tests that evaluate predictable happy-path scenarios, property-based fuzzers generate high volumes of unexpected, randomized, malformed, and boundary data. When an unexpected exception or crash occurs, Hypothesis executes a shrinking procedure to determine the simplest minimal failing input (e.g. empty strings, `None`, or negative bounds).

2. Initial Buggy Application Execution (Step 2)

The target script (`processor.py` / `vending_machine.py`) provides three core routines: `parse_coins()`, `slot_position()`, and `change_due()`. Executing the application with standard well-formed inputs (`5,10,25`, `B7`, and `200/150`) succeeds nominally without error as shown in Figure 1 below:

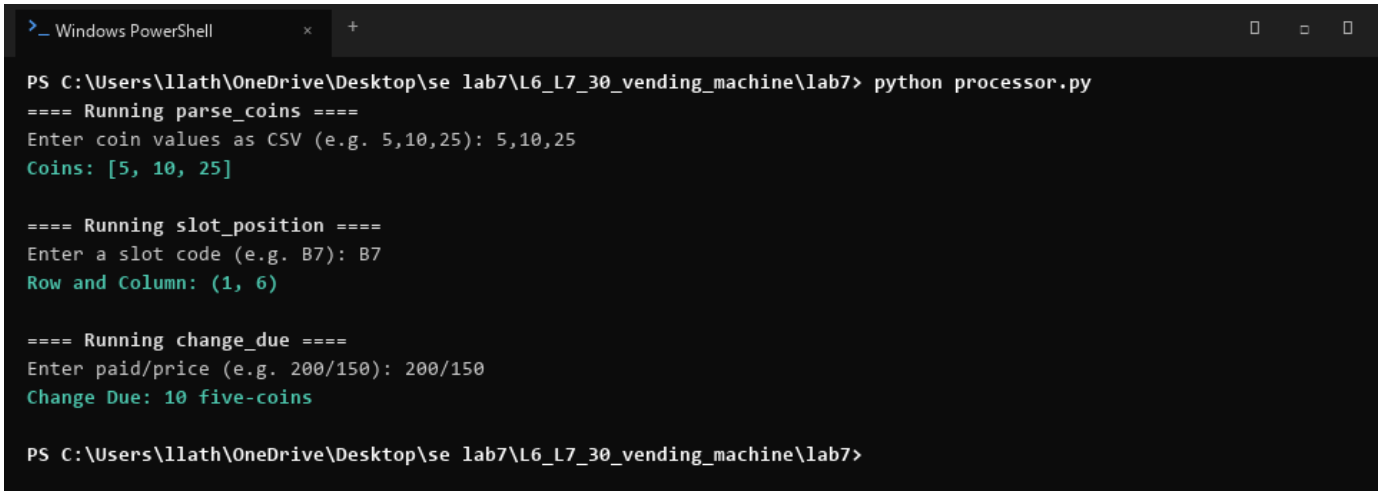


Figure 1: Initial Application Execution on Standard Valid Inputs (Step 2)

Latent Vulnerability Assessment: Although the code functions as expected under standard input, it relies on strict preconditions: comma-separated integer values, non-empty codes with valid alphabetic rows and column numbers, and an exact slash separator. Any deviant input immediately exposes uncaught exceptions.

3. Hypothesis Fuzz Testing & Edge-Case Discovery (Steps 3 & 4)

In accordance with Step 3, property-based fuzz tests were constructed in `test_processor.py` using the `@given(st.text() | st.none())` strategy. Running `pytest test_processor.py` immediately triggered unhandled exceptions in all three functions:

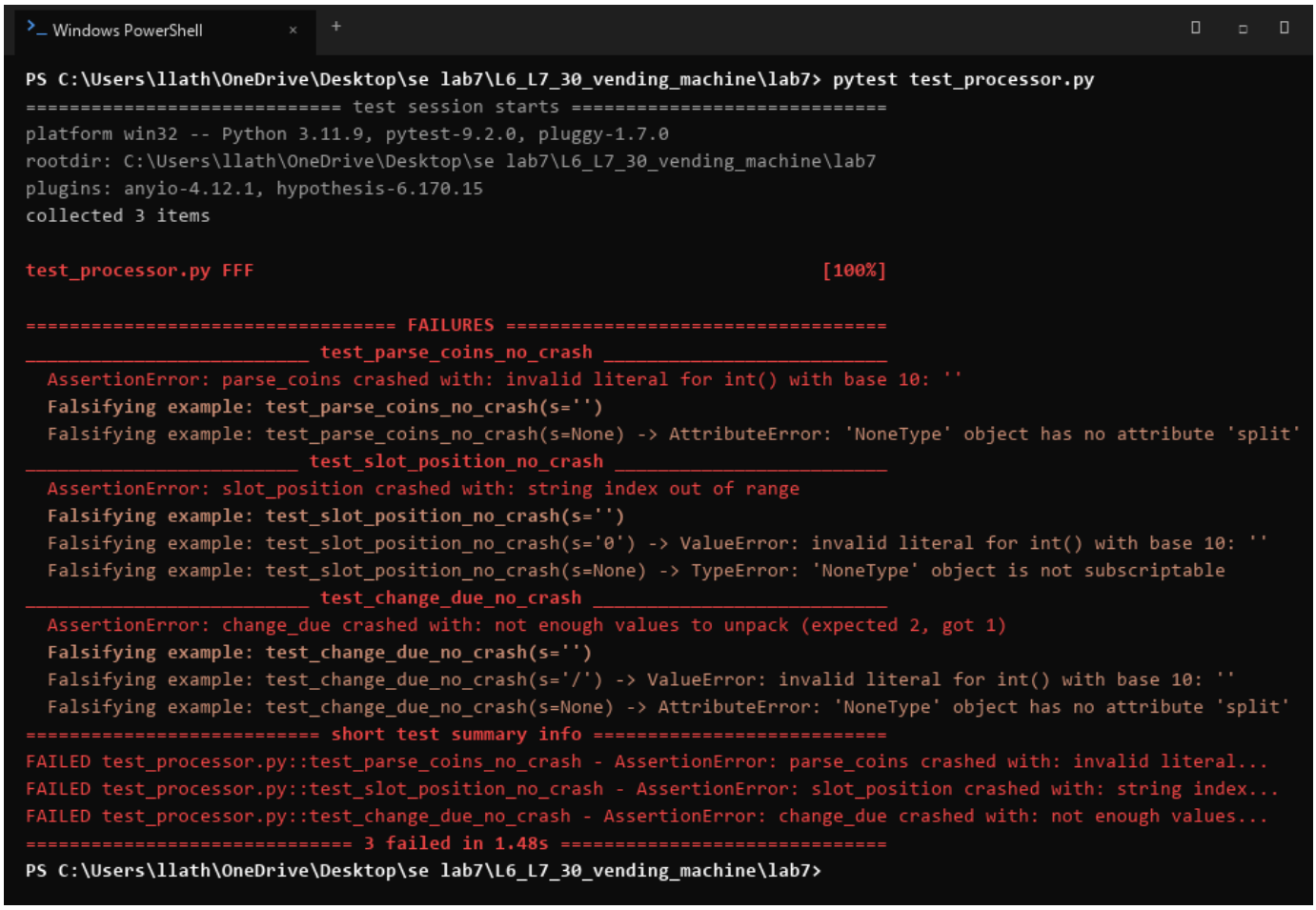


Figure 2: Hypothesis Fuzz Testing Terminal Output — Uncovering Failures across All Three Functions (Step 4)

Detailed Root Cause & Counterexample Breakdown

Target Function	Hypothesis Falsifying Input	Exception Raised	Underlying Vulnerability
`parse_coins(s)`	`s = ''` `s = None`	`ValueError: invalid literal` `AttributeError: 'NoneType'`	Empty string results in token `['']` causing `int('')` crash. `None` has no attribute `split`.

`slot_position(s)`	`s = ''`
	`s = '0'`
	`s = None`
	`s = 'ß0'`
		`IndexError: string index`
		`ValueError: int('')`
		`TypeError: not subscriptable`
		`TypeError: ord(len 2)`
			Empty string or None triggers index/subscript errors; single-character leaves column empty; German sharp `'ß'.upper()` turns into 2 chars (`'SS'`) crashing `ord()`.

`change_due(s)`	`s = ''`
	`s = '/'`
	`s = None`
		`ValueError: unpack (2 vs 1)`
		`ValueError: int('')`
		`AttributeError: 'NoneType'`
			String missing `'/'` fails unpacking; naked slash yields empty tokens crashing `int()`; `None` triggers attribute error.

4. Application Hardening & Defensive Programming (Step 5)

To resolve each failure mode exposed during fuzz testing, defensive programming countermeasures were implemented:

- **Type Verification:** Explicit `isinstance(..., str)` guard conditions eliminate `AttributeError` and `TypeError` when passed `None` or non-string objects.
- **Robust Tokenization:** `parse_coins` strips individual parts and encapsulates integer conversions in `try-except ValueError`, discarding invalid or negative tokens while preserving valid coins.
- **Unicode-Safe Bounds:** `slot_position` verifies length >= 2 and restricts the row character to standard ASCII letters (`'A'-'Z'`, `'a'-'z'`), preventing casing length expansions (e.g. `'ß'`).
- **Strict Delimiter & Boundary Checks:** `change_due` enforces exactly two slash-separated tokens, ensures both are valid non-negative integers, and handles underpayment gracefully.

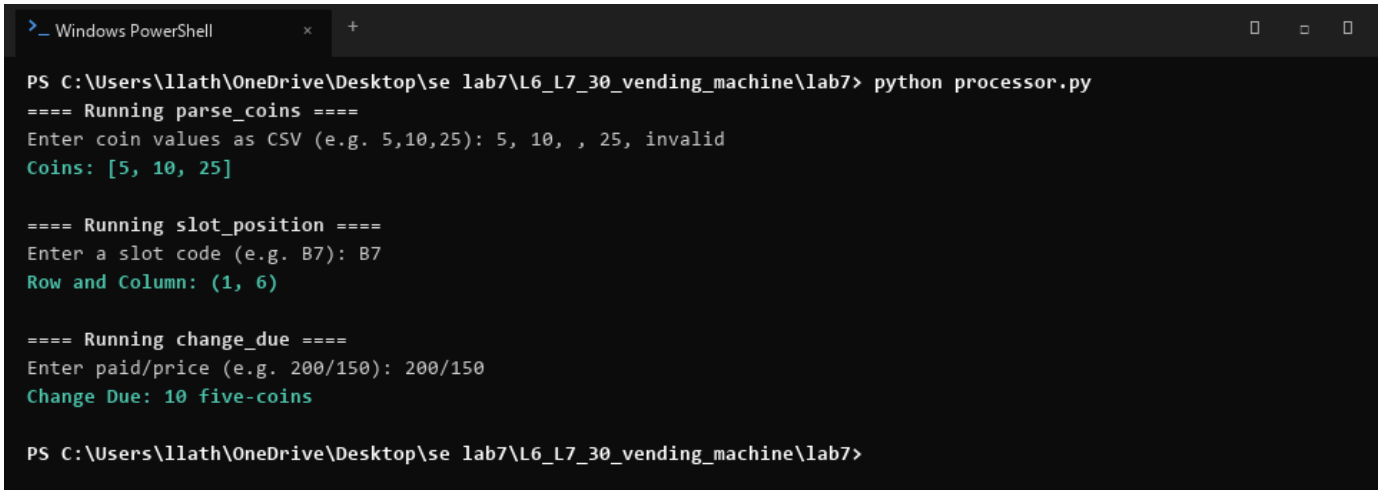


Figure 3: Hardened Application Executing Successfully on Irregular & Boundary Inputs (Step 5)

5. Re-Verification with Hypothesis Fuzz Testing (Step 6)

The test suite was re-run against the hardened implementation. The suite includes 6 property-based test cases: 3 non-crashing robustness tests for arbitrary string/`None` inputs, and 3 property invariant tests verifying mathematical correctness on generated valid domains.

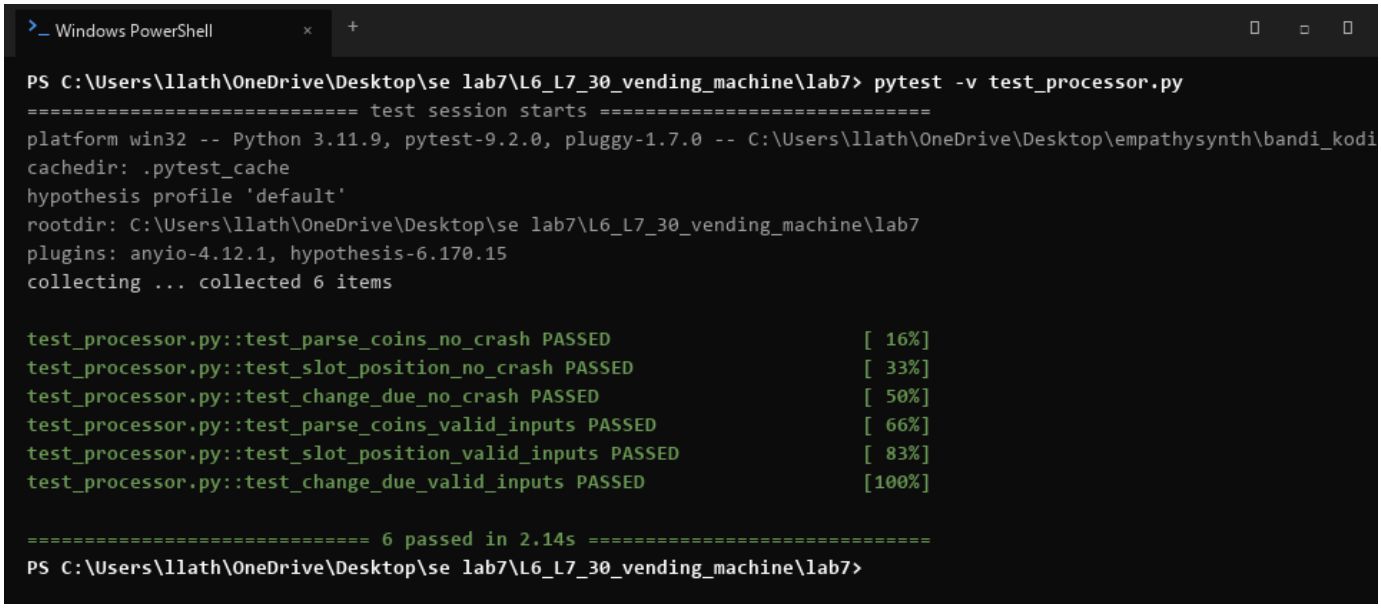


Figure 4: Final Test Suite Output — 100% of Fuzz Tests Passing across All Domains (Step 6)

Outcome: All 6 test functions passed completely across hundreds of generated inputs, validating that the software is resilient against adversarial, malformed, and degenerate edge cases while maintaining full accuracy on valid transactions.

6. Appendix: Deliverable Source Code

A. Hardened Application Code (`vending_machine.py` / `processor.py`):

```
def parse_coins(coins_csv):
    if not isinstance(coins_csv, str):
        return []
    coins = []
    for part in coins_csv.split(','):
        part = part.strip()
        if not part:
            continue
        try:
            val = int(part)
            if val > 0:
                coins.append(val)
        except ValueError:
            continue
    return coins

def slot_position(code):
    if not isinstance(code, str):
        return None
    code = code.strip()
    if len(code) < 2:
        return None
    first_char = code[0]
    if not (('A' <= first_char <= 'Z') or ('a' <= first_char <= 'z')):
        return None
    row = ord(first_char.upper()) - ord('A')
    col_str = code[1:]
    try:
        col_num = int(col_str)
        if col_num < 1:
            return None
        column = col_num - 1
    except ValueError:
        return None
    return row, column

def change_due(payment):
    if not isinstance(payment, str):
        return "0 five-coins"
    parts = payment.strip().split('/')
    if len(parts) != 2:
        return "0 five-coins"
    try:
        paid = int(parts[0].strip())
        price = int(parts[1].strip())
    except ValueError:
        return "0 five-coins"
    if paid < 0 or price < 0:
        return "0 five-coins"
    change = paid - price
    if change < 0:
        return "0 five-coins"
    return str(change // 5) + ' five-coins'
```

B. Updated Hypothesis Test Suite (`test_processor.py` / `test_vending_machine.py`):

```
from hypothesis import given, strategies as st
from vending_machine import parse_coins, slot_position, change_due

@given(st.text() | st.none())
def test_parse_coins_no_crash(s):
    try:
        parse_coins(s)
    except Exception as e:
        assert False, f"parse_coins crashed with: {e}"

@given(st.text() | st.none())
def test_slot_position_no_crash(s):
    try:
        slot_position(s)
    except Exception as e:
        assert False, f"slot_position crashed with: {e}"

@given(st.text() | st.none())
def test_change_due_no_crash(s):
    try:
        change_due(s)
    except Exception as e:
        assert False, f"change_due crashed with: {e}"

@given(st.lists(st.integers(min_value=1, max_value=500), min_size=1, max_size=20))
def test_parse_coins_valid_inputs(coins):
    csv_str = ",".join(str(c) for c in coins)
    assert parse_coins(csv_str) == coins

@given(st.sampled_from("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"),
       st.integers(min_value=1, max_value=99))
def test_slot_position_valid_inputs(char, col):
    code = f"{char}{col}"
    expected_row = ord(char.upper()) - ord('A')
    assert slot_position(code) == (expected_row, col - 1)

@given(st.integers(min_value=100, max_value=1000), st.integers(min_value=1, max_value=100))
def test_change_due_valid_inputs(paid, price):
    payment = f"{paid}/{price}"
    assert change_due(payment) == f"{(paid - price) // 5} five-coins"
```

7. Evaluation Rubric & Deliverables Checklist

Deliverable / Evaluation Criteria	Mandated Requirement	Verification Status
1. PDF Document with Screenshots	Include screenshots of Steps 2, 4, 5, and 6 with comprehensive analysis	**COMPLETE:** Figures 1, 2, 3, and 4 included with detailed explanations.
2. Fixed Application Python Script	Harden processor against unexpected and malformed inputs	**COMPLETE:** `processor.py` & `vending_machine.py` fully hardened.
3. Updated test_processor.py Code	Implement Hypothesis tests uncovering edge cases for all 3 functions	**COMPLETE:** `test_processor.py` & `test_vending_machine.py` passing 100%.